

quantchdb: A Well-Encapsulated ClickHouse Database APIs Lib

Quick Start

Install quantchdb:

```
pip install quantchdb==0.1.11 -i https://pypi.org/simple
```

An example of how to use quantchdb:

1. Import quantchdb

```
from quantchdb import ClickHouseDatabase
import pandas as pd
import numpy as np
```

2. Configure ClickHouseDatabase instance

```
# To connect your clickhouse database, you need to setup your config, in which the
'.env' method is recommended for security
config = {
    "host": os.getenv("DB_HOST", "localhost"),
    "port": int(os.getenv("DB_PORT", 9000)),
    "user": os.getenv("DB_USER", "default"),
    "password": os.getenv("DB_PASSWORD", ""),
    "database": os.getenv("DB_DATABASE", "default")
}

# 'terminal_log' and 'file_log' control the log records. 'True' denotes the
corresponding log method will be executed. You can control the logs' file path by
the 'log_file' param.
db = ClickHouseDatabase(config=config, terminal_log=True, file_log=False)
```

3. Functions

```
# Fetch data from clickhouse database
sql = "SELECT * FROM stocks.snap ORDER BY date DESC LIMIT 5"
df = db.fetch(sql)

# Execute SQL sentence
sql = f"""
```

```

CREATE TABLE IF NOT EXISTS etf.kline_1m(
    `exg` UInt8 NOT NULL COMMENT '交易所标识, 沪市为1, 深市为0, 北交所为2',
    `code` String NOT NULL COMMENT '股票代码',
    `date` Date NOT NULL COMMENT '日期',
    `date_time` DateTime('Asia/Shanghai') NOT NULL COMMENT '日期时间, 最高精度为秒',
    `time_int` UInt32 NOT NULL COMMENT '从当日开始至当前时刻的毫秒数',
    `open` Float32 NULL COMMENT 'K线开始价格',
    `high` Float32 NULL COMMENT 'K线内最高价',
    `low` Float32 NULL COMMENT 'K线内最低价',
    `close` Float32 NULL COMMENT 'K线结束价格',
    `volume` UInt64 NULL COMMENT 'K线内成交量',
    `amount` Float32 NULL COMMENT 'K线内成交额'
)Engine = ReplacingMergeTree()
ORDER BY (code, date_time);
"""

db.execute(sql)

# Insert dataframe into clickhouse database. Before you insert your dataframe, you
# need to make sure the corresponding database and table are existed.
# Make sure the dtypes of DataFrame is consistent with dtypes of clickhouse table,
# or else insert_dataframe may failed.

file_path = "Your/Data/Path/kline_1m.csv"
dtype_dict = {
    'exg' : np.int,
    'code' : str,
    'open' : np.float32,
    'close' : np.float32,
    'high' : np.float32,
    'low' : np.float32,
    'amount' : np.float32
}
df = pd.read_csv(file_path, dtype=dtype_dict)

#Int type with NA need to deal with seperately
df['volume'] = pd.to_numeric(df['volume'], errors='coerce').astype('UInt64')

db.insert_dataframe(
    df=df,
    table_name="etf.kline_1m",
    datetime_cols=['date', 'date_time'],
    convert_tz=False
)

# Create table from DataFrame and insert data into table automatically. This
# method is not recommended, because data type inferred may be not suitable or even
# the sentence failed.

# You can use dtypes to make sure some columns have corrected dtypes and use other
# params to control the create sql sentence, though dtypes/engine/orderby/other have
# default values.

db.create_table_from_df(df=df,

```

```
table_name='test.etf_kline_1m',  
dtypes={'code': 'String',  
        'date': 'Date',  
        'date_time' : 'DateTime'},  
engine='ReplacingMergeTree()',  
orderby='(code,date_time)',  
other='PARTITION BY toYYYYMM(code)')
```